

Flask lecture 1



Flask

Flask introduction

- Introduction
- Simple Flask application
- Flask slug value
- Flask HTTP methods
- HTML render with Flask application



Flask introduction

- Flask is a web framework for Python. It is classified as a microframework because it does not require particular tools or libraries. It has a small and easy-to-extend core. It is a WSGI (Web Server Gateway Interface) web application framework. It is often used for building APIs, simple web apps, and prototyping. Flask is lightweight and easy to learn, making it a popular choice for beginners and small projects.
- We will see that for a deeper understanding of Flask we will need to use all the previous tools that we learned in the course.

Flask introduction continue

- In order to understand Flask we need to understand several terminologies:
 - WSGI Web Server Gateway Interface (WSGI) has been adopted as a standard for Python web application development. WSGI is a specification for a universal interface between the web server and the web applications.
 - Werkzeug It is a WSGI toolkit, which implements requests, response objects, and other utility functions. This enables building a web framework on top of it. The Flask framework uses Werkzeug as one of its bases.
 - jinja2 jinja2 is a popular templating engine for Python. A web templating system combines a template with a certain data source to render dynamic web pages.
- The first thing that we need to do when we are starting to create a Flask application is to install Flask module. `pip install Flask`

Flask introduction continue

When we download a Flask module we also download additional modules into our Python project.

```
(venv) (base) alexk@alexk-Latitude-7420:~/PycharmProjects/flask_lecture$ pip freeze
```

```
click==8.1.3
```

```
Flask==2.2.2
```

```
importlib-metadata==6.0.0
```

```
itsdangerous==2.1.2
```

```
Jinja2==3.1.2
```

```
MarkupSafe==2.1.1
```

```
Werkzeug==2.2.2
```

```
zipp==3.11.0
```

```
(venv) (base) alexk@alexk-Latitude-7420:~/PycharmProjects/flask_lecture$ pip show Flask
```

```
Name: Flask
```

```
Version: 2.2.2
```

```
Summary: A simple framework for building complex web applications.
```

```
Home-page: https://palletsprojects.com/p/flask
```

```
Author: Armin Ronacher
```

```
Author-email: armin.ronacher@active-4.com
```

```
License: BSD-3-Clause
```

```
Location: /home/alexk/PycharmProjects/flask_lecture/venv/lib/python3.8/site-packages
```

```
Requires: click, importlib-metadata, itsdangerous, Jinja2, Werkzeug
```

```
Required-by:
```

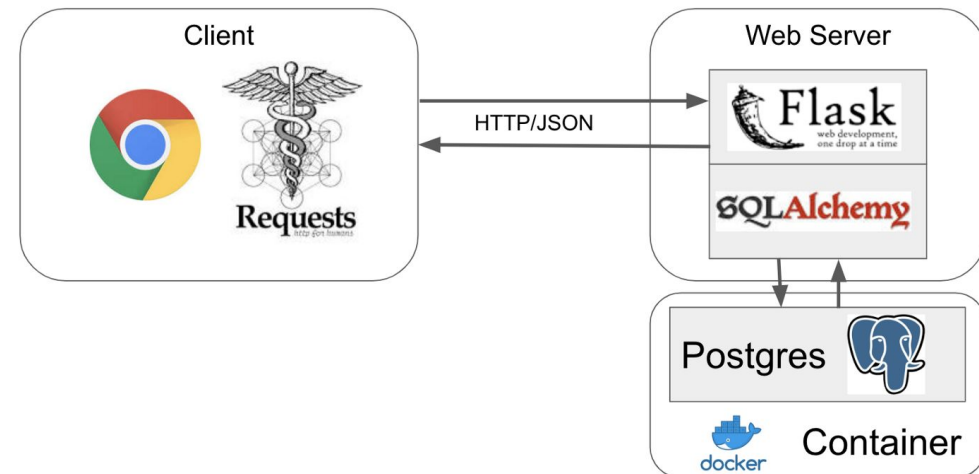
Flask simple application

We will create a simple Flask application in order to understand basic Flask architecture.

```
simple_application.py x
1  # Importing flask module in the project is mandatory
2  # An object of Flask class is our WSGI application.
3  from flask import Flask
4  # Flask constructor takes the name of
5  # current module (__name__) as argument.
6  app = Flask(__name__)
7  # The route() function of the Flask class is a decorator,
8  # which tells the application which URL should call
9  # the associated function.
10 @app.route('/')
11 # '/' URL is bound with hello_world() function.
12 def hello_world():
13     return 'Hello World'
14 # main driver function
15 if __name__ == '__main__':
16     # run() method of Flask class runs the application
17     # on the local development server.
18     app.run()
```



Hello World



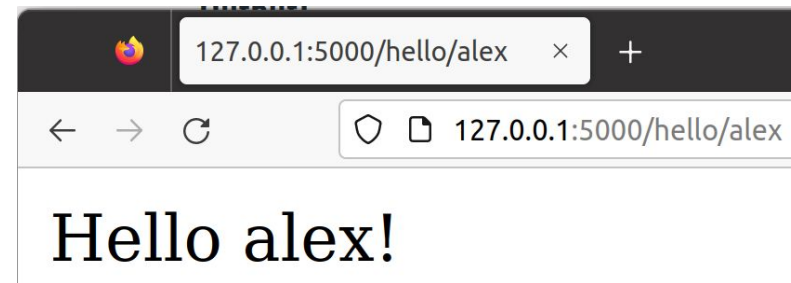
Flask slug value

- Think that for each page we will need to have a different URL, how we will be able to handle it dynamically? For it we also can handle our URL as a variable. This value is called slug. Here is an example.
- In Flask, a "slug" is a string that can be used in the URL of a webpage to identify a specific resource or page. It is commonly used in web applications to uniquely identify a particular item or object. A slug is typically a short, human-readable string that is derived from the title or name of an object and is used in the URL to refer to that object. For example, a blog post with the title "Hello, World!" might have a slug "hello-world" and the URL to access that post would be something like "www.example.com/blog/hello-world".
- You can use the `url_for()` function in Flask to generate URLs based on the slugs, so that you don't have to hard-code URLs in your templates. It helps to keep the URLs cleaner and more readable.

Flask slug value continue

Here is a code example.

```
slug_example.py x
1  from flask import Flask
2  app = Flask(__name__)
3  @app.route('/hello/<name>')
4  def hello_name(name):
5      return 'Hello %s!' % name
6  if __name__ == '__main__':
7      app.run()
8
```



Flask HTTP methods

- We can also use HTTP methods in Flask let's see how to do that.
- The HTTP protocol is the foundation of data communication on the world wide web. Different methods of data retrieval from specified URL are defined in this protocol. The methods are described down below.
 - **GET** : Sends data in simple or unencrypted form to the server.
 - **HEAD** : Sends data in simple or unencrypted form to the server without body.
 - **HEAD** : Sends form data to the server. Data is not cached.
 - **PUT** : Replaces target resource with the updated content.
 - **DELETE** : Deletes target resource provided as URL.
- By default, the Flask route responds to the GET requests. However, this preference can be altered by providing methods argument to route() decorator. In order to demonstrate the use of the POST method in URL routing, first, let us create an HTML form and use the POST method to send form data to a URL. Now let's create an HTML login page.

Flask HTTP methods continue

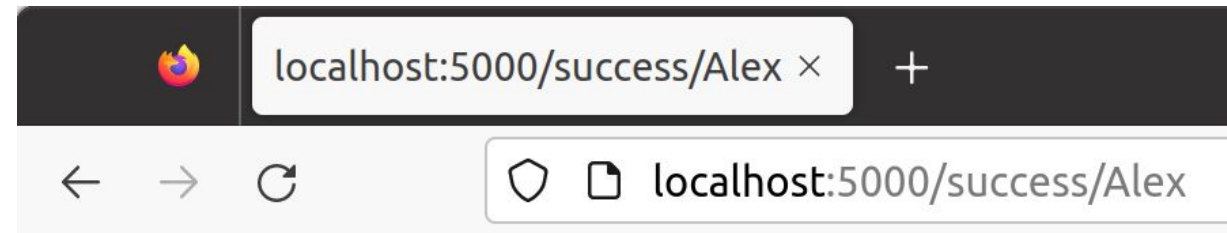
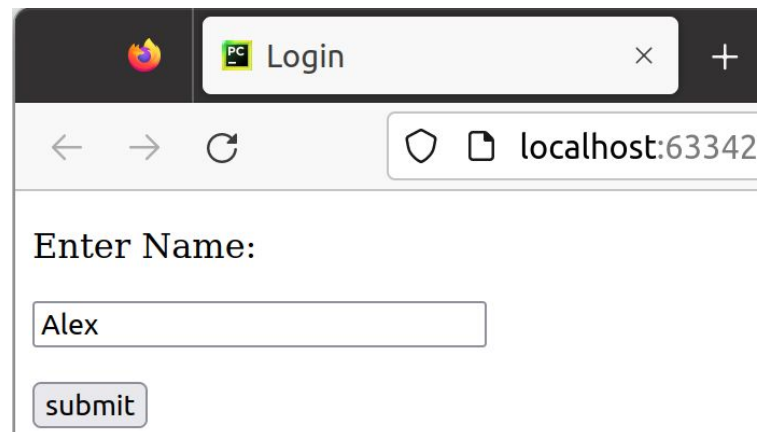
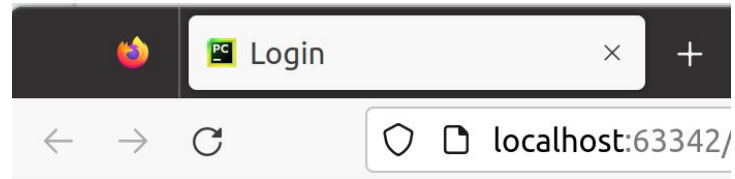
The code of our Login HTML page and our login Flask server.

```
login_server.py x login.html x
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Login</title>
6 </head>
7 <body>
8     <form action = "http://localhost:5000/login" method = "post">
9         <p>Enter Name:</p>
10        <p><input type = "text" name = "nm" /></p>
11        <p><input type = "submit" value = "submit" /></p>
12    </form>
13 </body>
14 </html>

login_server.py x
1 from flask import Flask, redirect, url_for, request
2 app = Flask(__name__)
3 @app.route('/success/<name>')
4 def success(name):
5     return 'welcome %s' % name
6 @app.route('/login', methods=['POST', 'GET'])
7 def login():
8     if request.method == 'POST':
9         user = request.form['nm']
10        return redirect(url_for('success', name=user))
11    else:
12        user = request.args.get('nm')
13        return redirect(url_for('success', name=user))
14 if __name__ == '__main__':
15     app.run(debug=True)
```

Flask HTTP methods continue

The communication of our server and our HTML page result.



welcome Alex

Flask HTTP methods continue

The code of our Login HTML page and our login Flask server.

```
login_server.py x login.html x
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Login</title>
6 </head>
7 <body>
8     <form action = "http://localhost:5000/login" method = "post">
9         <p>Enter Name:</p>
10        <p><input type = "text" name = "nm" /></p>
11        <p><input type = "submit" value = "submit" /></p>
12    </form>
13 </body>
14 </html>

login_server.py x
1 from flask import Flask, redirect, url_for, request
2 app = Flask(__name__)
3 @app.route('/success/<name>')
4 def success(name):
5     return 'welcome %s' % name
6 @app.route('/login', methods=['POST', 'GET'])
7 def login():
8     if request.method == 'POST':
9         user = request.form['nm']
10        return redirect(url_for('success', name=user))
11    else:
12        user = request.args.get('nm')
13        return redirect(url_for('success', name=user))
14 if __name__ == '__main__':
15     app.run(debug=True)
```

Flask HTML rendering

- When we are starting using HTML files in Flask we must create a dedicated Project structure for a successful rendering.

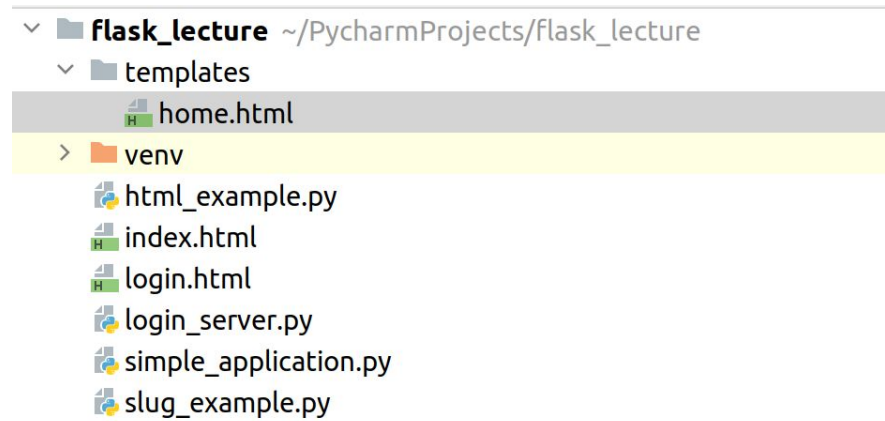
myapp

|

|- templates

|

|- home.html

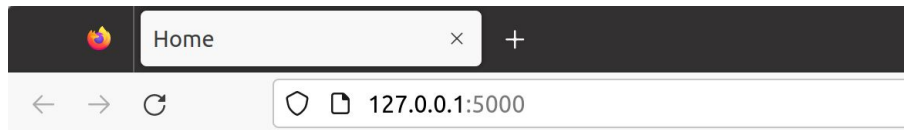


- In this example, the home function is mapped to the "/" route, so when a user navigates to the root of the website, the home.html template will be rendered and displayed to the user. The render_template function takes the name of the template file as its first argument and any additional arguments are passed to the template as variables.

Flask HTML rendering continue

Now let's see the code and run it to see the result for now.

```
html_example.py x home.html x
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def home():
7     return render_template("home.html")
8
9 if __name__ == "__main__":
10     app.run(debug=True)
```



```
html_example.py x home.html x
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Home</title>
5 </head>
6 <body>
7     <h1>Welcome Alex</h1>
8 </body>
9 </html>
```

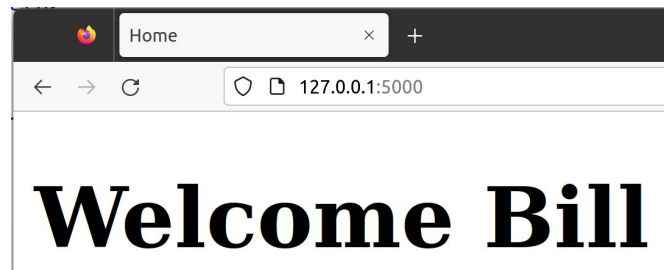
Welcome Alex

Flask HTML rendering continue

We can transfer a variable value into an HTML from Flask application. The syntax that will be used in the HTML is called Jinja2. If you remember this is one of the modules that were downloaded with Flask download.

```
html_example.py x home.html x
1  from flask import Flask, render_template
2
3  app = Flask(__name__)
4
5  @app.route("/")
6  def home():
7      name = "Bill"
8      return render_template("home.html", name = name)
9
10 if __name__ == "__main__":
11     app.run(debug=True)
```

```
html_example.py x home.html x
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Home</title>
5  </head>
6  <body>
7      <h1>Welcome {{ name }}</h1>
8  </body>
9  </html>
```



Exercise



- Create a dedicated directory for your Flask application.
- Create a templates directory.
- Implement the HTTP methods example completely with HTML's.

Solution part 1



- ✓ flask_lecture ~/PycharmProjects/flask_lecture
 - ✓ exercise
 - ✓ templates
 - login.html
 - welcome_page.html
 - app.py

```
app.py x
1  from flask import Flask, redirect, url_for, request, render_template
2  app = Flask(__name__)
3  @app.route('/success/<name>')
4  def success(name):
5      return render_template("welcome_page.html", name = name)
6  @app.route('/login', methods=['POST', 'GET'])
7  def login():
8      if request.method == 'POST':
9          user = request.form['nm']
10         return redirect(url_for('success', name=user))
11     else:
12         user = request.args.get('nm')
13         return redirect(url_for('success', name=user))
14  @app.route('/')
15  def home():
16      return render_template("login.html")
17  if __name__ == '__main__':
18      app.run(debug=True)
```

Solution part 2



```
app.py x login.html x
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Login</title>
6 </head>
7 <body>
8   <form action = "http://localhost:5000/login" method = "post">
9     <p>Enter Name:</p>
10    <p><input type = "text" name = "nm" /></p>
11    <p><input type = "submit" value = "submit" /></p>
12  </form>
13 </body>
14 </html>
```

Enter Name:


```
welcome_page.html x
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Welcome</title>
5 </head>
6 <body>
7   <h1>Welcome {{ name }}</h1>
8 </body>
9 </html>
```

Welcome Tom

Flask continue

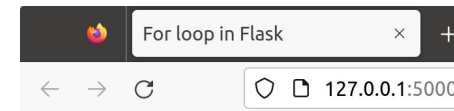
- For loop in HTML in Flask
- Static files



For loop in HTML with Flask

- Flask is one of the web development frameworks written in Python. Through flask, a loop can be run in the HTML code using jinja template and automatically HTML code can be generated using this.
- The code of app.py is same for both examples. We will print a Python list with Some names of Pokemons first in the format of a list and then a table.

```
app.py x
1  # importing modules
2  from flask import Flask, render_template
3  # declaring app name
4  app = Flask(__name__)
5  # making list of pokemons
6  Pokemons = ["Pikachu", "Charizard", "Squirtle", "Jigglypuff",
7              "Bulbasaur", "Gengar", "Charmander", "Mew", "Lugia", "Gyarados"]
8  # defining home page
9  @app.route('/')
10 def homepage():
11     # returning index.html and list
12     # and length of list to html page
13     return render_template("index.html", len=len(Pokemons), Pokemons=Pokemons)
14 if __name__ == '__main__':
15     app.run(use_reloader=True, debug=True)
16
```



1. Pikachu
2. Charizard
3. Squirtle
4. Jigglypuff
5. Bulbasaur
6. Gengar
7. Charmander
8. Mew
9. Lugia
10. Gyarados

For loop in HTML with Flask

- Flask is one of the web development frameworks written in Python. Through flask, a loop can be run in the HTML code using jinja template and automatically HTML code can be generated using this.
- The code of app.py is same for both examples. We will print a Python list with Some names of Pokemons first in the format of a list and then a table.

```
app.py x
1  # importing modules
2  from flask import Flask, render_template
3  # declaring app name
4  app = Flask(__name__)
5  # making list of pokemons
6  Pokemons = ["Pikachu", "Charizard", "Squirtle", "Jigglypuff",
7              "Bulbasaur", "Gengar", "Charmander", "Mew", "Lugia", "Gyarados"]
8  # defining home page
9  @app.route('/')
10 def homepage():
11     # returning index.html and list
12     # and length of list to html page
13     return render_template("index.html", len=len(Pokemons), Pokemons=Pokemons)
14 if __name__ == '__main__':
15     app.run(use_reloader=True, debug=True)
16
```



1. Pikachu
2. Charizard
3. Squirtle
4. Jigglypuff
5. Bulbasaur
6. Gengar
7. Charmander
8. Mew
9. Lugia
10. Gyarados

For loop in HTML with Flask continue

We will use the argument Pokemons passed from python file here to automatically print a table instead of Writing it our self. Code for app.py for this example is same as the above one.

```
index.html x app.py x
1 # importing modules
2 from flask import Flask, render_template
3 # declaring app name
4 app = Flask(__name__)
5 # making list of pokemons
6 Pokemons = ["Pikachu", "Charizard", "Squirtle", "Jigglypuff",
7             "Bulbasaur", "Gengar", "Charmander", "Mew", "Lugia", "Gyarados"]
8 # defining home page
9 @app.route('/')
10 def homepage():
11     # returning index.html and list
12     # and length of list to html page
13     return render_template("index.html", len=len(Pokemons), Pokemons=Pokemons)
14 if __name__ == '__main__':
15     app.run(use_reloader=True, debug=True)
```

```
index.html x
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>For loop in Flask</title>
5 </head>
6 <style type="text/css">
7     th:tr{
8         color: blue;
9     }
10    tr:nth-of-type(2n){
11        border: 1px solid black;
12        background-color: rgba(150, 150, 150, 0.5);
13    }
14    td{
15        padding: 8px 8px;
16        border: 1px solid black;
17    }
18 </style>
19 <body>
20 <table style="margin-left: 20px;">
21 <!-- Table headers -->
22 <tr>
23     <th style="color: green;">
24         <td>Serial Number</td>
25         <td>Pokemon Name</td>
26     </tr>
27 </th>
28 {%for i in range(0, len)%}
29 <tr>
30     <td>{{i}}</td>
31     <td>{{Pokemons[i]}}</td>
32 {%endfor%}
33 </tr>
34 </table>
35 </body>
36 </html>
```

For loop in Flask	
127.0.0.1:5000	
Serial Number	Pokemon Name
0	Pikachu
1	Charizard
2	Squirtle
3	Jigglypuff
4	Bulbasaur
5	Gengar
6	Charmander
7	Mew
8	Lugia
9	Gyarados

For loop in HTML with Flask continue

If we want to include css file we need to create a directory called static and link it with the command in the example.

- for_loop_html2
 - static
 - mystyle.css
 - templates
 - index.html
 - app.py

```
index.html x
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>For loop in Flask</title>
5 </head>
6 <link href="{{ url_for('static', filename='mystyle.css') }}" rel="stylesheet">
7 </style>
8 <body>
9 <table style="margin-left: 20px;">
10  <!-- Table headers -->
11  <tr>
12    <th style="color: green;">
13      <td>Serial Number</td>
14      <td>Pokemon Name</td>
15  </tr>
16  </th>
17  {%for i in range(0, len)%}
18  <tr>
19    <td>{{i}}</td>
20    <td>{{Pokemons[i]}}</td>
21  {%endfor%}
22  </tr>
23 </table>
24 </body>
25 </html>
```

Serial Number	Pokemon Name
0	Pikachu
1	Charizard
2	Squirtle
3	Jigglypuff
4	Bulbasaur
5	Gengar
6	Charmander
7	Mew
8	Lugia
9	Gyarados

Exercise



- Create an interactive web application that will get signup information from a user.
- The application will have an additional page that will present all the signed up users.

Thank you for your time

